LAPORAN PRAKTIKUM

STRUKTUR DATA

“Abstract Data Type”

disusun oleh:

Dennis Shauqi Akbar

2511533020

Dosen pengampu: Dr. Wahyudi. S.T.M.T

Asisten Pratikum: Sherly Sukmadira

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

TAHUN 2025

KATA PENGANTAR

Puji dan syukur kami panjatkan ke hadirat Allah SWT atas segala rahmat dan karunia Nya, sehingga kami dapat menyelesaikan tugas kelompok ini dengan baik. Shalawat serta salam semoga senantiasa tercurah kepada junjungan kita Nabi Muhammad SAW, keluarga, sahabat, serta para pengikutnya hingga akhir zaman. Tugas Individu ini kami susun dengan tujuan untuk memenuhi salah satu tugas mata kuliah Struktur Data, dengan Judul "Pemrograman GUI 2" yang mencakup Kalkulator.

Kami menyadari bahwa penyusunan tugas ini masih jauh dari sempurna, baik dari segi materi maupun penyajiannya. Oleh karena itu, kami sangat mengharapkan kritik dan saran yang membangun dari semua pihak demi kesempurnaan tugas kami di masa mendatang.

Akhir kata, kami mengucapkan terima kasih kepada dosen pembimbing yang telah memberikan arahan sehingga tugas ini dapat terselesaikan tepat waktu. Semoga tugas ini dapat bermanfaat bagi kami khususnya, dan bagi pembaca pada umumnya.

Padang, 02 April 2026

Dennis Shauqi Akbar

DAFTAR ISI

KATA PENGANTAR 1

DAFTAR ISI 2

BAB I PENDAHULUAN 3

1.1 Latar Belakang 3

1.2 Tujuan 3

1.3 Manfaat 2

BAB II PEMBAHASAN 1

2.1 Pengertian ADT 1

2.2 Implementasi pada Java 4

2.2.1 Class Inti (Jam_2511533020) 4

2.2.2 Class Driver 1 (JamDriver1_2511533020) 6

2.2.3 Class Driver 2 JamDriver2_2511533020 7

BAB III PENUTUP 12

3.1 Kesimpulan 12

3.2 Saran 12

DAFTAR PUSTAKA 11

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam mata kuliah Struktur Data, pemahaman tentang cara menyimpan dan mengelola data secara efisien merupakan hal yang fundamental. Pada praktikum kali ini, saya mempelajari konsep Abstract Data Type (ADT) yang berfungsi sebagai kerangka logis untuk mendefinisikan tipe data beserta operasi-operasi yang dapat dilakukan terhadap data tersebut, tanpa perlu memikirkan detail implementasi teknisnya.

Latar belakang pemilihan topik ini adalah karena ADT menjadi pondasi penting dalam pengembangan perangkat lunak, terutama untuk memisahkan antara antarmuka (interface) dan implementasi internal. Dengan demikian, ADT memudahkan perancangan program yang modular, reusable, serta mudah dipelihara karena menerapkan prinsip enkapsulasi.

Melalui praktikum ini, saya bertujuan untuk mengimplementasikan dan menguji secara langsung konsep ADT menggunakan bahasa pemrograman yang telah ditentukan. Pemahaman yang baik terhadap ADT akan sangat berguna nantinya saat saya membangun struktur data yang lebih kompleks seperti stack, queue, tree, atau graph

1.2 Tujuan

Tujuan dari praktikum ini adalah untuk membangun dan mengimplementasikan Abstract Data Type (ADT) Jam yang merepresentasikan waktu dengan atribut jam, menit, dan detik secara terenkapsulasi. Saya juga bertujuan untuk menerapkan

validasi data, konstruktor, selektor, mutator, serta berbagai operasi seperti konversi ke/dari detik, perbandingan antar jam, aritmatika waktu (penjumlahan, pengurangan, maju mundur detik), dan penghitungan durasi. Selain itu, melalui dua kelas driver (sederhana dan berbasis input pengguna), saya menguji seluruh fungsionalitas ADT tersebut sekaligus belajar menangani input yang tidak valid secara interaktif. Pada akhir praktikum, saya diharapkan mampu memahami dan menerapkan konsep ADT dalam pemrograman berorientasi objek menggunakan bahasa Java

1.3 Manfaat

Melalui praktikum ini, saya memperoleh manfaat berupa pemahaman konkret tentang implementasi Abstract Data Type (ADT) dalam bahasa Java, khususnya bagaimana membangun kelas Jam yang terenkapsulasi dengan atribut private serta menyediakan metode selektor, mutator, dan validasi. Saya juga belajar melakukan konversi antara representasi objek dan total detik, membandingkan dua objek waktu dengan compareTo, serta melakukan operasi aritmatika waktu seperti penjumlahan, pengurangan, dan maju mundur detik dengan penanganan wrap around 24 jam. Selain itu, pembuatan dua driver (sederhana dan interaktif) melatih saya dalam menguji program secara sistematis sekaligus menangani input pengguna yang tidak valid. Semua keterampilan ini sangat berguna untuk pengembangan aplikasi berbasis waktu di masa mendatang, seperti sistem penjadwalan, manajemen event, atau simulasi waktu.

BAB II

PEMBAHASAN

2.1 Pengertian ADT

Abstract Data Type (ADT) atau Tipe Data Abstrak merupakan konsep dalam pemrograman yang mendefinisikan tipe data berdasarkan perilaku (operasi-operasi yang dapat dilakukan) dan bukan berdasarkan implementasi teknisnya. Dengan kata lain, ADT memisahkan antara "apa yang bisa dilakukan" terhadap suatu data dengan "bagaimana cara melakukannya". Contoh sederhana dalam kehidupan sehari-hari adalah tipe data `Jam` yang saya buat pada praktikum ini; pengguna hanya perlu tahu bahwa ia dapat mengambil jam, menit, detik, atau menjumlahkan dua jam, tanpa perlu tahu bagaimana detail internal kode tersebut bekerja.

Salah satu karakteristik utama ADT adalah enkapsulasi, yaitu menyembunyikan detail internal suatu objek dari pengguna luar. Pada kode program saya, atribut `hh`, `mm`, `ss` dideklarasikan sebagai `private`, sehingga tidak dapat diakses langsung dari luar kelas. Akses hanya diberikan melalui metode publik seperti `getHour()`, `setHour()`, `plus()`, `toSeconds()`, dan sebagainya. Dengan cara ini, jika suatu saat saya ingin mengubah representasi internal (misalnya menyimpan waktu dalam bentuk total detik saja), saya dapat melakukannya tanpa mengganggu program lain yang menggunakan ADT `Jam` ini, selama antarmuka publiknya tetap sama.

Dalam konteks praktikum Struktur Data, pemahaman ADT menjadi fondasi penting sebelum mempelajari struktur data yang lebih kompleks seperti stack,

queue, tree, atau graph. ADT yang saya praktikkan pada kelas `Jam_2511533020` merupakan contoh konkret bagaimana data dienkapsulasi, divalidasi, dan diberikan berbagai operasi (konstruktor, selektor, mutator, relasional, aritmatika) sesuai kebutuhan. Dengan menguasai ADT, saya dapat merancang program yang lebih modular, mudah diuji, dan mudah dipelihara karena setiap komponen bertanggung jawab pada satu tugas spesifik.

2.2 Implementasi pada Java

Pada praktikum pekan 1 mahasiswa disuruh untuk mengikuti intruksi dari dosen. Yaitu menuliskan kembali kode program yang sudah ada di papan tulis. Kode program yang dibuat tentang jam, di bawah ini kode program yang dituliskan pada praktikum pekan 1.

2.2.1 Class Inti (Jam_2511533020)

```java
package pekan1_2511533020;

public final class Jam_2511533020 {
    private int hh; // 0..23
    private int mm; // 0..59
    private int ss; // 0..59

    // ---Validator ---
    public static boolean isValid(int h, int m, int s) {
        return (0 <= h && h <= 23) && (0 <= m && m <= 59) && (0 <= s && s <= 59);
    }

    // --- Kontruktor/Kreator ---
    public Jam_2511533020(int h, int m, int s) {
        this.hh = h; this.mm = m; this.ss = s;

    }

    // --- Selektor ---
    public int getHour() { return hh; }
    public int getMinute() { return mm; }
    public int getSecond() { return ss; }

    // --- Mutator ---
    public void setHour(int h) { this.hh = h; }
    public void setMinute(int m) { this.mm = m; }
    public void setSecond(int s) { this.ss = s; }

    // --- Konversi ---
    public int toSeconds() { return hh * 3600 + mm * 60 + ss; }
    public static Jam_2511533020 fromSeconds (int total) {
        if (total < 0) throw new IllegalArgumentException("detik negatif");
        total %= 24 * 3600; // wrap 24 jam
        int h = total / 3600; total %= 3600;
        int m = total / 60; int s = total % 60;
        return new Jam_2511533020(h, m, s);
    }
```

```java
    // --- Relasional ---
    public int compareTo (Jam_2511533020 other) { return Integer.compare(this.toSeconds(), other.toSeconds()); }
    public boolean equals(Object o) {
        if (! (o instanceof Jam_2511533020 j)) return false;
        return hh == j.hh && mm == j.mm && ss == j.ss;
    }
    public int hashCode() { return java.util.Objects.hash(hh, mm, ss); }

    // --- Aritmatika ---
    public Jam_2511533020 plus(Jam_2511533020 other) { return fromSeconds(this.toSeconds() + other.toSeconds()); }
    public Jam_2511533020 minus(Jam_2511533020 other) { return fromSeconds(Math.floorMod(this.toSeconds() - other.toSeconds
    public Jam_2511533020 nextSecond() { return fromSeconds(this.toSeconds() + 1); }
    public Jam_2511533020 nextNSeconds(int n) { return fromSeconds(this.toSeconds() + Math.max(0,  n)); }
    public Jam_2511533020 prevSecond() { return fromSeconds(Math.floorMod(this.toSeconds () - 1, 24*3600));}
    public Jam_2511533020 PrevNSecond(int n) { return fromSeconds(Math.floorMod(this.toSeconds() - Math.max(0,  n), 24*3600

    // Durasi (detik) -- bisa negatif jika this > other? sesuai spesifikasi durasi(JAw, JAkh)
    public static int durasiDetik(Jam_2511533020 jaw, Jam_2511533020 jakh) { return jakh.toSeconds() - jaw.toSeconds(); }
    //overriding ke string
    public String toString() { return String.format("%02d:%02d:%02d", hh, mm, ss); }

}
```

Gambar 2.2.1

Pada kelas Jam_2511533020, terdapat tiga variabel yaitu hh untuk jam, mm untuk menit, dan ss untuk detik. Ketiga variabel ini diberi tanda private, yang berarti tidak bisa diakses atau diubah secara langsung dari luar kelas. Selain itu, tersedia method isValid() untuk memeriksa apakah nilai jam, menit, dan detik yang dimasukkan sudah benar, yaitu jam antara 0 sampai 23, menit dan detik antara 0 sampai 59.

Selanjutnya, terdapat konstruktor Jam_2511533020(int h, int m, int s) yang digunakan untuk membuat objek jam baru. Ada pula method getHour(), getMinut e(), getSecond() yang berfungsi mengambil nilai masing masing variabel, serta set Hour(), setMinute(), setSecond() untuk mengubah nilainya. Method konversi toSe conds() mengubah jam menjadi total detik, sedangkan fromSeconds() mengubah total detik kembali ke bentuk jam. Method fromSeconds() menggunakan operasi sisa bagi (%) agar hasilnya tetap dalam rentang satu hari (24 jam). Jika total detik negatif, method ini akan memunculkan pesan kesalahan.

Pada bagian akhir kelas, ditambahkan method untuk membandingkan dua jam (compareTo dan equals), method aritmatika seperti plus(), minus(), nextSecond(), prevSecond(), nextNSeconds(), PrevNSecond(), serta method durasiDetik() untuk menghitung selisih detik antara dua jam. Semua method aritmatika tersebut memanfaatkan toSeconds() dan fromSeconds() agar perhitungan lebih mudah. Ter akhir, method toString() yang dicetak ulang (override) membuat tampilan objek jam menjadi format HH:MM:SS dengan dua digit.

2.2.2 Class Driver 1 (JamDriver1_2511533020)

```java
package pekan1_2511533020;

import java.util.Scanner;

public class JamDriver2_2511533020 {

    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        System.out.println("=== Program Driver Objek Jam===");
        // 1. Input Jam Pertama
        System.out.println("\n--- Input Jam_2511533020 1 ---");
        Jam_2511533020 j1 = buatJamDariInput(input);

        // 2. Input Jam Kedua
        System.out.println("\n--- Input Jam_2511533020 2 ---");
        Jam_2511533020 j2 = buatJamDariInput(input);

        // 3. Menampilkan Data
        System.out.println("\n--- Hasil Operasi---");
        System.out.println("Jam_2511533020 1 (String)        : " + j1.toString());
        System.out.println("Jam_2511533020 2 (String)        : " + j2.toString());
        System.out.println("Jam_2511533020 1 dalam detik     : " + j1.toSeconds());
        System.out.println("Jam_2511533020 2 dalam detik     : " + j2.toSeconds());

        //4. Operasi Relasional (Perbandingan)
        int perbandingan = j1.compareTo(j2);
        if (perbandingan > 0) {
            System.out.println("Status                :Jam_2511533020 1 lebih lambat (setelah) Jam_2511533020 2");

        } else if (perbandingan < 0) {
            System.out.println("Status                :Jam_2511533020 1 lebih awal (sebelum) Jam_2511533020 2");
        } else {
            System.out.println("Status                :Jam_2511533020 1 dan Jam_2511533020 2 sama persis");
        }

        // 5. Operasi Aritmatika
        System.out.println("Durasi (J1 ke J2)       : " + Jam_2511533020.durasiDetik(j1, j2) + " detik ");

```

Gambar 2.2.2

Pada kelas JamDriver_2511533020 terdapat method main yang menjadi titik awal jalannya program. Di dalam method ini, dibuat dua buah objek dari kelas Jam_2511533020 yaitu objek a dengan nilai waktu 23:59:50 dan objek b dengan nilai waktu 00:00:15. Kedua objek tersebut langsung dicetak ke layer menggunakan perintah System.out.println. Karena kelas Jam_2511533020 s

udah memiliki method toString(), maka tampilan yang muncul adalah format HH:MM:SS seperti "23:59:50" dan "00:00:15".

Selanjutnya, program memanggil beberapa method yang sudah tersedia di kelas Jam_2511533020. Method a.plus(b) digunakan untuk menjumlahkan dua waktu, sehingga hasilnya adalah 23:59:50 ditambah 00:00:15 menjadi 00:00:05 (karena terjadi kelebihan satu hari yang langsung diwrap). Method a.nextNSecond s(20) membuat waktu a maju sebanyak 20 detik, sehingga hasilnya menjadi 00:00:10. Method durasiDetik(a, b) menghitung selisih detik dari a ke b, yaitu dari pukul 23:59:50 ke 00:00:15 yang hasilnya adalah 25 detik.

Terakhir, program memanggil method a.compareTo(b) untuk membandingkan objek a dan b. Karena a lebih besar dari b (23:59:50 lebih lambat dari 00:00:15), maka hasil yang dikembalikan adalah angka positif. Semua hasil dari setiap pemanggilan method tersebut langsung dicetak ke layar. Driver ini berguna untuk menguji fungsionalitas dasar ADT Jam secara cepat tanpa perlu berinteraksi dengan pengguna.

2.2.3 Class Driver 2 JamDriver2_2511533020

```java
package pekan1_2511533020;

import java.util.Scanner;

public class JamDriver2_2511533020 {

    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        System.out.println("=== Program Driver Objek Jam===");
        // 1. Input Jam Pertama
        System.out.println("\n--- Input Jam_2511533020 1 ---");
        Jam_2511533020 j1 = buatJamDariInput(input);

        // 2. Input Jam Kedua
        System.out.println("\n--- Input Jam_2511533020 2 ---");
        Jam_2511533020 j2 = buatJamDariInput(input);

        // 3. Menampilkan Data
        System.out.println("\n--- Hasil Operasi---");
        System.out.println("Jam_2511533020 1 (String)        : " + j1.toString());
        System.out.println("Jam_2511533020 2 (String)        : " + j2.toString());
        System.out.println("Jam_2511533020 1 dalam detik     : " + j1.toSeconds());
        System.out.println("Jam_2511533020 2 dalam detik     : " + j2.toSeconds());

        //4. Operasi Relasional (Perbandingan)
        int perbandingan = j1.compareTo(j2);
        if (perbandingan > 0) {
            System.out.println("Status                :Jam_2511533020 1 lebih lambat (setelah) Jam_2511533020 2");

        } else if (perbandingan < 0) {
            System.out.println("Status                :Jam_2511533020 1 lebih awal (sebelum) Jam_2511533020 2");
        } else {
            System.out.println("Status                :Jam_2511533020 1 dan Jam_2511533020 2 sama persis");
        }

        // 5. Operasi Aritmatika
        System.out.println("Durasi (J1 ke J2)       : " + Jam_2511533020.durasiDetik(j1, j2) + " detik ");

```

```java
        Jam_2511533020 jNext = j1.nextSecond();
        System.out.println("Jam_2511533020 1 Detik Berikutnya: " + jNext);

        Jam_2511533020 jPrev = j1.prevSecond();
        System.out.println("Jam_2511533020 1 Detik Sebelumnya: " + jPrev);

        // 6. Operasi Penjumlahan Jam
        Jam_2511533020 jHasilPlus = j1.plus(j2);
        System.out.println("Hasil J1 + J2                : " + jHasilPlus);

        input.close();
        System.out.println("\nProgram Selesai.");

    }

```

Pada kelas JamDriver2_2511533020, terdapat method main yang menggun akan Scanner untuk menerima input dari pengguna. Program terlebih dahulu menampilkan judul "=== Program Driver Objek Jam===". Selanjutnya, program memanggil method buatJamDariInput sebanyak dua kali untuk membuat dua objek jam yaitu j1 dan j2. Method buatJamDariInput bekerja dengan meminta pengguna memasukkan nilai jam, menit, dan detik secara berulang. Jika nilai yang dimasukkan tidak valid (misalnya jam lebih dari 23), program akan menampilkan pesan error dan meminta input ulang hingga benar. Validasi ini menggunakan method isValid() dari kelas Jam_2511533020.

Setelah kedua objek berhasil dibuat, program menampilkan berbagai hasil operasi. Pertama, kedua objek ditampilkan dalam bentuk string menggunakan toString(), kemudian masing-masing dikonversi ke total detik menggunakan toSeconds(). Selanjutnya, program membandingkan j1 dan j2 denga n compareTo(). Jika hasil perbandingan positif, maka j1 lebih lambat dari j2; jika negatif, j1 lebih awal; jika nol, kedua waktu sama persis. Program juga menghitung durasi detik dari j1 ke j2 menggunakan method durasiDetik().

Terakhir, program menjalankan beberapa operasi aritmatika waktu. Method nextSecond() menghasilkan waktu satu detik setelah j1, sedangkan prevS econd() menghasilkan satu detik sebelumnya. Method plus() menjumlahkan j1 da n j2, lalu hasilnya ditampilkan. Setelah semua operasi selesai, Scanner ditutup dan program menampilkan pesan "Program Selesai." Driver interaktif ini berguna untuk menguji ADT Jam secara lebih lengkap karena pengguna dapat memasukkan nilai sendiri dan melihat hasil operasi secara langsung.

BAB III

PENUTUP

3.1 Kesimpulan

Berdasarkan praktikum Abstract Data Type (ADT) yang telah dilakukan, dapat disimpulkan bahwa ADT merupakan konsep penting dalam pemrograman berorientasi objek yang memisahkan data dari operasi yang dapat dilakukan terhadap data tersebut, seperti yang terlihat pada kelas Jam_2511533020 dengan variabel private dan method publik seperti konstruktor, selektor, mutator, serta operasi aritmatika dan relasional. Pengujian menggunakan dua driver (sederhana dan interaktif) membuktikan bahwa semua method berfungsi dengan baik, termasuk validasi input menggunakan isValid() yang memastikan hanya nilai waktu sah yang diproses. Dengan demikian, pemahaman ADT sangat berguna sebagai fondasi untuk mempelajari struktur data yang lebih kompleks seperti stack, queue, tree, atau graph.

3.2 Saran

Untuk pengembangan lebih lanjut, disarankan agar konstruktor pada kelas Jam_2511533020 dilengkapi dengan pemanggilan method isValid() sehingga nilai yang tidak valid dapat ditolak sejak pembuatan objek. Selain itu, method nextNSeconds dan PrevNSecond sebaiknya tidak hanya menerima nilai positif tetapi juga nilai negatif agar lebih fleksibel. Pada driver interaktif, disarankan untuk menambahkan opsi keluar (exit) jika pengguna ingin menghentikan program di tengah jalan, serta menangani kemungkinan input selain angka (misalnya huruf) agar program tidak mudah error. Terakhir, praktikum

berikutnya dapat menambahkan fitur seperti konversi ke format waktu 12 jam (AM/PM) atau perhitungan durasi dalam menit dan jam secara terpisah.

DAFTAR PUSTAKA

[1] N. Dale and H. M. Walker, *Abstract Data Types: Specifications, Implementations, and Applications*. Lexington, MA, USA: D.C. Heath, 1996.

[2] Oracle, "Class Object (Java Platform SE 17)," OpenJDK. [Online]. Available: https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/Object.html (diakses: 2 April 2026).

[3] D. I. Gunawan Hts et al., *Struktur Data dan Algoritma: Pondasi Komputasi Modern. Teori dan Praktik dengan Java*. Indonesia: Penerbit Teknosain, 2025.

[4] Wikipedia, "Tipe data abstrak," Wikipedia. [Online]. Available: https://id.wikipedia.org/wiki/Tipe_data_abstrak (diakses: 2 April 2026)

LAPORAN TUGAS PEKAN 1

SOAL:

Tugas Praktikum Pekan 1

- Buatlah sebuah program ADT mobil yang memiliki atribut: nama, tahun, cc, harga, dan merk
- Contoh: Avanza, 2020, 1300, 150000000, Toyota
- Buatlah method ADT mobil tersebut: tambah mobil, hapus mobil, Selektor dan mutator
- Buatlah 2 kelas: 1 kelas ADT mobil dan 1 kelas driver. Setiap kelas lengkapi dengan nim

a) Class 1 (Mobil_2511533020)

```java
package tugaspekan1_2511533020;

public class Mobil_2511533020 {
    private String nama;
        private int tahun;
        private int cc;
        private double harga;
        private String merek;

        // Konstruktor
        public Mobil_2511533020(String nama, int tahun, int cc, double harga, String merek) {
            this.nama = nama;
            this.tahun = tahun;
            this.cc = cc;
            this.harga = harga;
            this.merek = merek;
        }

        // Selektor (getter)
        public String getNama() { return nama; }
        public int getTahun() { return tahun; }
        public int getCc() { return cc; }
        public double getHarga() { return harga; }
        public String getMerek() { return merek; }

        // Mutator (setter)
        public void setNama(String nama) { this.nama = nama; }
        public void setTahun(int tahun) { this.tahun = tahun; }
        public void setCc(int cc) { this.cc = cc; }
        public void setHarga(double harga) { this.harga = harga; }
        public void setMerek(String merek) { this.merek = merek; }

        // Method untuk menampilkan data mobil
        public String toString() {
            return merek + " " + nama + ", Tahun " + tahun + ", " + cc + "cc, Rp" + harga;
        }
    }
```

Pada kelas `Mobil_2511533020`, terdapat lima atribut yaitu `nama` (String), `tahun` (int), `cc` (int), `harga` (double), dan `merek` (String). Kelima atribut ini diberi tanda `private`, sehingga tidak bisa diakses atau diubah langsung dari luar kelas.

Hal ini bertujuan agar data mobil hanya bisa dimanipulasi melalui method-method yang sudah disediakan. Kelas ini juga memiliki sebuah konstruktor `Mobil_2511533020(String nama, int tahun, int cc, double harga, String merek)` yang digunakan untuk membuat objek mobil baru sekaligus mengisi nilai atributnya.

Selanjutnya, kelas ini menyediakan method selektor (getter) untuk mengambil nilai masing masing atribut, yaitu `getNama()`, `getTahun()`, `getCc()`, `getHarga()`, dan `getMerek()`. Selain itu, disediakan juga method mutator (setter) seperti `setNama()`, `setTahun()`, `setCc()`, `setHarga()`, dan `setMerek()` yang berfungsi untuk mengubah nilai atribut setelah objek dibuat. Dengan adanya getter dan setter, kontrol terhadap data tetap terjaga karena setiap perubahan harus melalui method yang jelas.

Terakhir, kelas ini memiliki method `toString()` yang dicetak ulang (override) agar ketika objek mobil ditampilkan, formatnya menjadi rapi seperti `"Toyota Avanza, Tahun 2020, 1300cc, Rp150000000"`. Method ini sangat membantu saat debugging atau saat menampilkan daftar mobil ke layar. Secara keseluruhan, kelas `Mobil_2511533020` sudah mewakili konsep ADT karena data disembunyikan dan operasi terhadap data hanya bisa dilakukan melalui method yang tersedia.

b) Class 2 (Driver Mobil_2511533020)

```java
package tugaspekan1_2511533020;

import java.util.ArrayList;
import java.util.Scanner;

public class DriverMobil_2511533020 {
    private static ArrayList<Mobil_2511533020> daftarMobil = new ArrayList<>();
    private static Scanner input = new Scanner(System.in);

    public static void main(String[] args) {
        int pilihan;
        do {
            System.out.println("\n=== PROGRAM MANAJEMEN MOBIL ===");
            System.out.println("1. Tambah Mobil");
            System.out.println("2. Hapus Mobil");
            System.out.println("3. Tampilkan Semua Mobil");
            System.out.println("4. Ubah Data Mobil (Mutator)");
            System.out.println("5. Cari Mobil (Selector)");
            System.out.println("0. Keluar");
            System.out.print("Pilihan: ");
            pilihan = input.nextInt();
            input.nextLine(); // bersihkan buffer

            switch (pilihan) {
                case 1 -> tambahMobil();
                case 2 -> hapusMobil();
                case 3 -> tampilkanSemua();
                case 4 -> ubahMobil();
                case 5 -> cariMobil();
                case 0 -> System.out.println("Terima kasih.");
                default -> System.out.println("Pilihan tidak valid.");
            }
        } while (pilihan != 0);
    }

    // Method tambah mobil
    public static void tambahMobil() {
        System.out.println("\n--- TAMBAH MOBIL ---");
```

```java
        String nama = input.nextLine();
        System.out.print("Tahun: ");
        int tahun = input.nextInt();
        System.out.print("CC: ");
        int cc = input.nextInt();
        System.out.print("Harga: ");
        double harga = input.nextDouble();
        input.nextLine();
        System.out.print("Merek (contoh: Toyota): ");
        String merek = input.nextLine();

        Mobil_2511533020 mobilBaru = new Mobil_2511533020(nama, tahun, cc, harga, merek);
        daftarMobil.add(mobilBaru);
        System.out.println("Mobil berhasil ditambahkan.");
    }

    // Method hapus mobil berdasarkan indeks atau nama
    public static void hapusMobil() {
        if (daftarMobil.isEmpty()) {
            System.out.println("Belum ada mobil.");
            return;
        }
        tampilkanSemua();
        System.out.print("Masukkan nomor urut mobil yang akan dihapus: ");
        int idx = input.nextInt() - 1;
        if (idx >= 0 && idx < daftarMobil.size()) {
            Mobil_2511533020 hapus = daftarMobil.remove(idx);
            System.out.println("Mobil " + hapus.getNama() + " berhasil dihapus.");
        } else {
            System.out.println("Nomor urut tidak valid.");
        }
    }

    // Menampilkan semua mobil
    public static void tampilkanSemua() {
        if (daftarMobil.isEmpty()) {
            System.out.println("Tidak ada data mobil.");
        } else {
            System.out.println("\n--- DAFTAR MOBIL ---");
            for (int i = 0; i < daftarMobil.size(); i++) {
                System.out.println((i+1) + ". " + daftarMobil.get(i).toString());
            }
        }
    }

    // Mutator: mengubah data mobil yang sudah ada
    public static void ubahMobil() {
        if (daftarMobil.isEmpty()) {
            System.out.println("Tidak ada mobil untuk diubah.");
            return;
        }
        tampilkanSemua();
        System.out.print("Pilih nomor mobil yang akan diubah: ");
        int idx = input.nextInt() - 1;
        if (idx < 0 || idx >= daftarMobil.size()) {
            System.out.println("Nomor tidak valid.");
            return;
        }
        Mobil_2511533020 m = daftarMobil.get(idx);
        System.out.println("Data saat ini: " + m.toString());
        System.out.println("Masukkan data baru (kosongkan jika tidak ingin mengubah):");

        input.nextLine(); // buffer
        System.out.print("Nama baru (" + m.getNama() + "): ");
        String nama = input.nextLine();
        if (!nama.isEmpty()) m.setNama(nama);

        System.out.print("Tahun baru (" + m.getTahun() + "): ");
        String tahunStr = input.nextLine();
        if (!tahunStr.isEmpty()) m.setTahun(Integer.parseInt(tahunStr));

        System.out.print("CC baru (" + m.getCc() + "): ");
        String ccStr = input.nextLine();
        if (!ccStr.isEmpty()) m.setCc(Integer.parseInt(ccStr));

        System.out.print("Harga baru (" + m.getHarga() + "): ");
        String hargaStr = input.nextLine();
        if (!hargaStr.isEmpty()) m.setHarga(Double.parseDouble(hargaStr));

        System.out.print("Merek baru (" + m.getMerek() + "): ");
        String merek = input.nextLine();
        if (!merek.isEmpty()) m.setMerek(merek);

        System.out.println("Data mobil berhasil diubah.");
    }

    // Selector: mencari mobil berdasarkan nama (contoh penggunaan selector)
    public static void cariMobil() {
        System.out.print("Masukkan nama mobil yang dicari: ");
        String cariNama = input.nextLine();
        boolean ditemukan = false;
        for (Mobil_2511533020 m : daftarMobil) {
            if (m.getNama().equalsIgnoreCase(cariNama)) {
                System.out.println("Ditemukan: " + m.toString());
                ditemukan = true;
            }
        }
        if (!ditemukan) System.out.println("Mobil dengan nama \"" + cariNama + "\" tidak ditemukan.");
    }
}
```

Pada kelas DriverMobil_2511533020, terdapat sebuah ArrayList bernama daftarMobil yang berfungsi untuk menyimpan Kumpulan objek Mobil_25115330 20. Juga terdapat Scanner untuk menerima input dari pengguna. Di dalam method main, program menampilkan menu pilihan yang berisi: tambah mobil, hapus mobil, tampilkan semua, ubah data (mutator), cari mobil (selector), dan keluar. Menu ini dijalankan berulang menggunakan perulangan do-while hingga pengguna memilih angka 0.

Setiap operasi diimplementasikan dalam method tersendiri. Method tamba hMobil() meminta pengguna memasukkan nama, tahun, cc, harga, dan merek, lalu membuat objek Mobil_2511533020 baru dan menambahkannya ke ArrayList. Me thod hapus Mobil() menampilkan semua mobil terlebih dahulu, kemudian meminta nomor urut mobil yang akan dihapus. Method tampilkanSemua() menampilkan seluruh isi ArrayList dengan bantuan method toString() dari kelas mobil.

Fitur yang berkaitan dengan **selector** diwujudkan dalam method cariMobil (), yaitu dengan meminta nama mobil yang dicari lalu menggunakan getNama() u ntuk membandingkan dengan setiap mobil dalam daftar. Fitur **mutator** diwujudkan dalam method ubahMobil(), di mana pengguna memilih mobil yang akan diubah, lalu data lama ditampilkan, kemudian pengguna dapat memasukkan nilai baru untuk setiap atribut. Method ubahMobil() memanggil setter seperti setNama(), set Tahun(), dan seterusnya untuk mengubah data. Dengan demikian, semua spesifikasi yang diminta (tambah, hapus, selector, mutator) sudah terpenuhi dalam driver ini.